

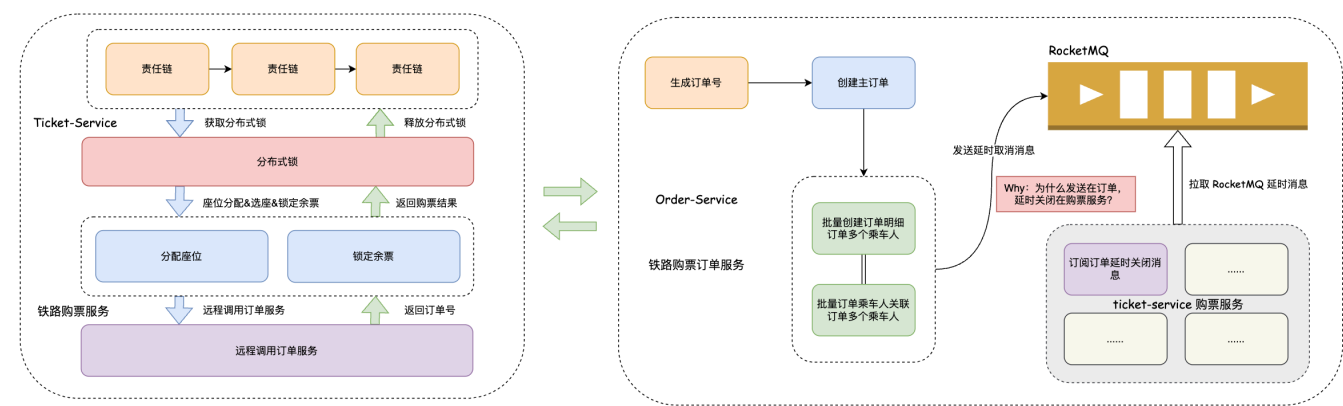
手把手之实现列车购票流程

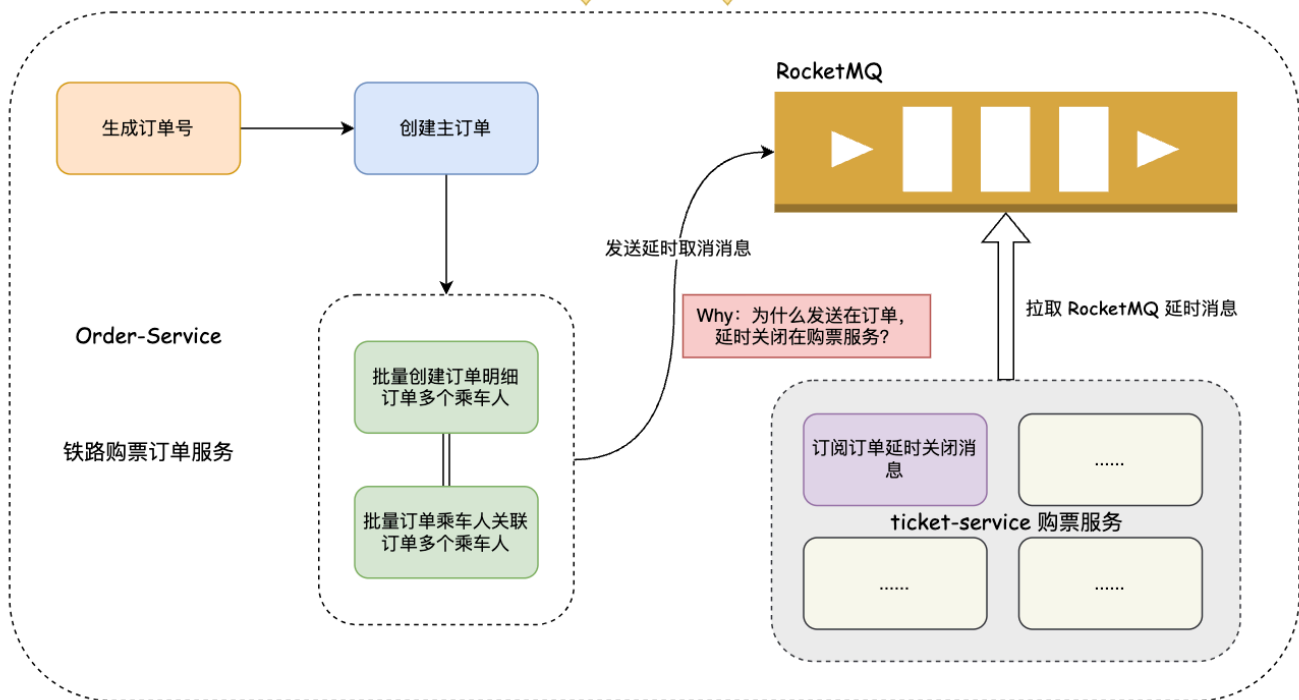
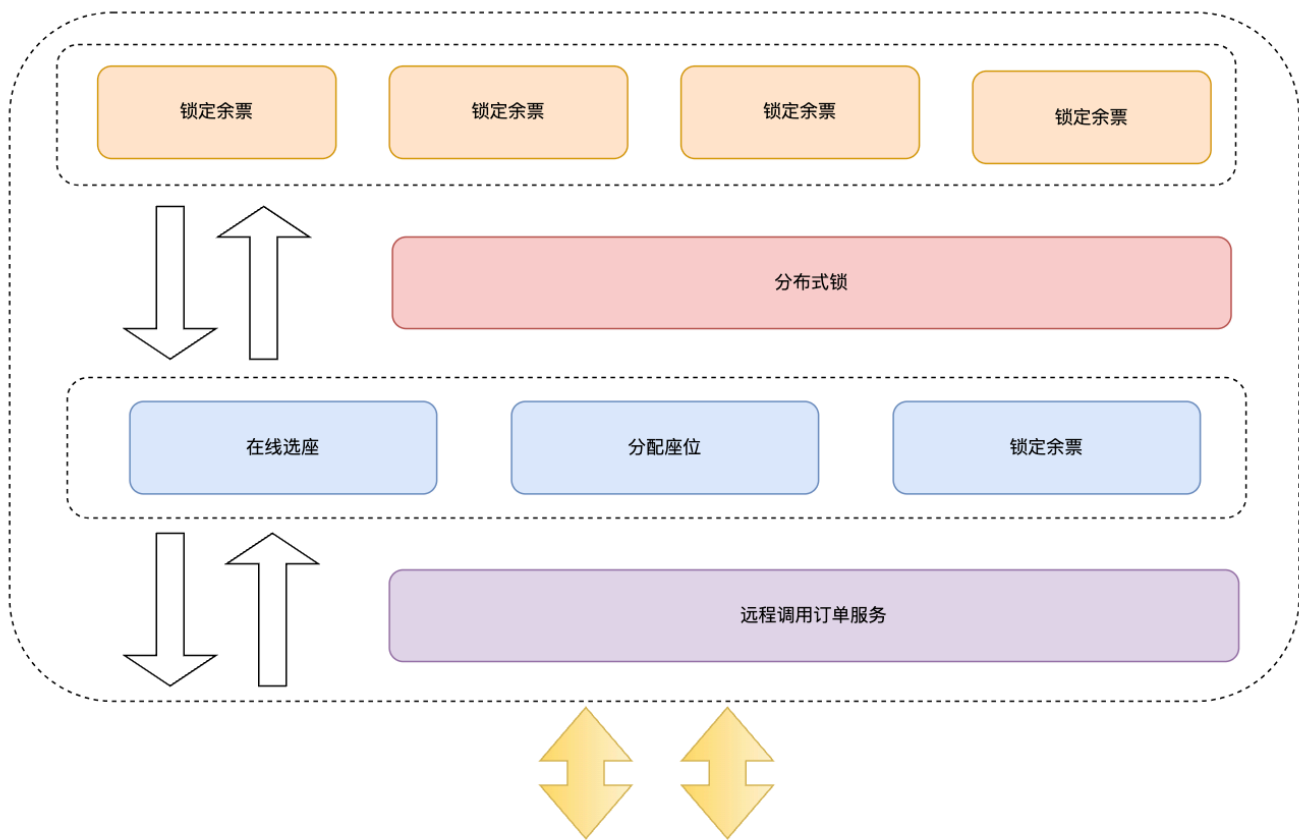
购票接口拆分为两个版本，分别是 v1 分布式锁版本以及 v2 令牌限流算法+分布式锁版本。

最早的分布式锁版本存在一些系统稳定性以及性能较低的问题，所以才因此升级了 v2 版本。为了让大家能更清晰的掌握两者的区别，所以代码中拆分了两个接口。

虽然 v2 版本要比 v1 版本具备更高的性能和稳定性，但是两个版本的购票核心逻辑是不变的。下文中咱们以 v1 分布式锁版本进行讲述。

购票流程





如何防重复提交

什么是防重复提交

HTTP 接口防重复提交是一种机制，用于防止在某个请求还未完成处理或响应返回之前，重复提交相同的请求。防重复提交的目的是确保每个请求只被处理一次，避免因重复提交而引起的数据重复处理、重复写入等问题。

如果不做防重复提交，可能会导致以下问题：

1. 数据重复处理：如果用户在某个操作还在处理中时重复提交请求，可能会导致相同的数据被处理多次，造成数据的重复操作和处理逻辑的错误。
2. 重复写入：在某些场景下，请求可能触发对数据库或其他存储系统的写入操作。如果请求被重复提交，可能会导致相同的数据被重复写入，破坏数据的一致性。
3. 资源浪费：重复提交请求可能会导致服务器资源的浪费。如果请求处理的时间较长，重复提交会占用服务器的处理能力，增加服务器的负载，降低系统的性能和吞吐量。
4. 业务逻辑错误：在某些业务场景下，重复提交可能会导致业务逻辑错误。例如，如果用户在生成订单的过程中重复提交订单请求，可能会导致生成多个相同的订单，引发订单的混乱和错误。

通过上述理论知识得知，如果购票接口不做防重复提交处理，有可能会触发一个用户无意创建多个订单的可能。为了防止这种行为，需要针对购票接口做 HTTP 防重复提交处理。

因为咱们针对 HTTP 的防重复提交和消息队列的幂等做了封装，所以直接引入组件库直接使用即可。

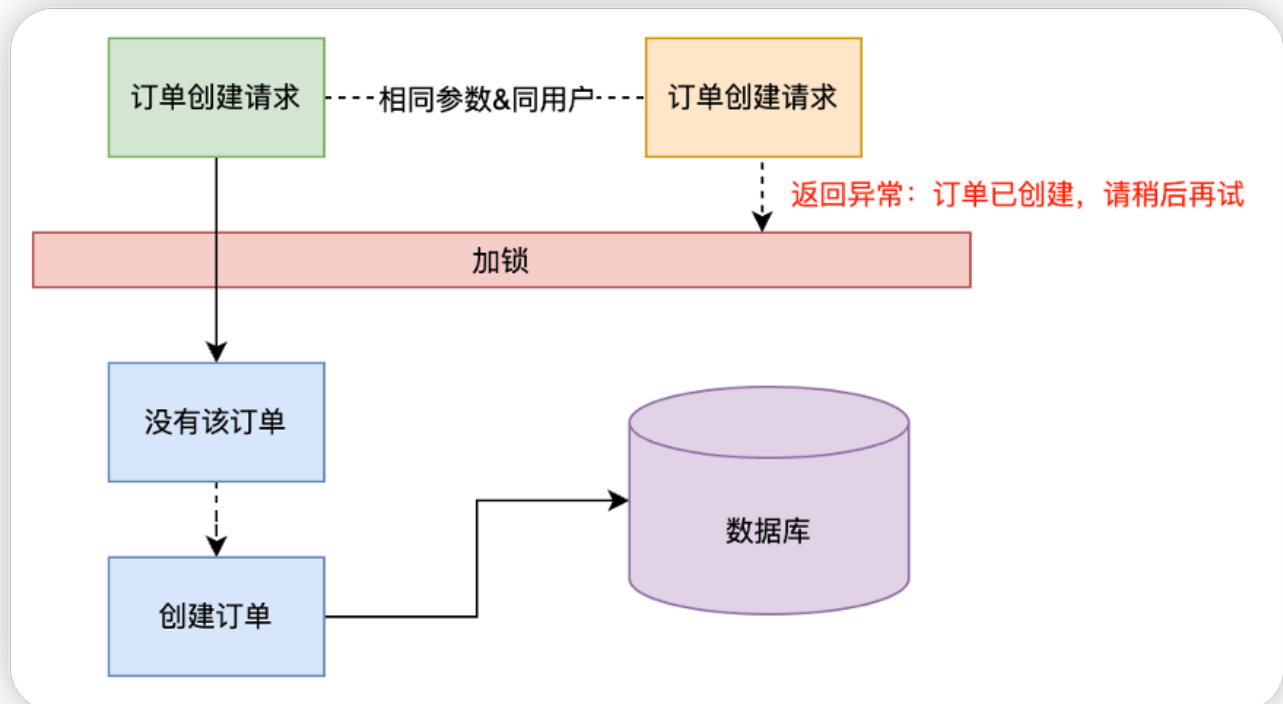
引入幂等组件

在购票服务中引入幂等组件 Maven Jar 坐标。

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-idempotent-spirng-boot-starter</artifactId>
  <!-- 因为购票服务和幂等组件库使用的都是一个版本，直接取购票服务版本号即可 -->
  <version>${project.version}</version>
</dependency>
```

接口添加防重复提交注解

添加防重复提交注解之前，思考一个问题，防重复提交的运行原理是什么？是不是加一把锁就能解决了，运行原理如下所示：



购票接口上添加组件中的防重复提交注解。

```
@Idempotent(  
    uniqueKeyPrefix = "index12306-ticket:lock_purchase-tickets:",  
    key =  
    "T(org.opengoofy.index12306.framework.starter.bases.ApplicationContextHolder).getBean('environment').getProperty('unique-name', '')"  
        + "'_'+"  
        +  
    "T(org.opengoofy.index12306.frameworks.starter.user.core.UserContext).getUsername()",  
    message = "正在执行下单流程，请稍后...",  
    scene = IdempotentSceneEnum.RESTAPI,  
    type = IdempotentTypeEnum.SPEL  
)  
@PostMapping("/api/ticket-service/ticket/purchase")  
public Result<TicketPurchaseRespDTO> purchaseTickets(@RequestBody PurchaseTicketReqDTO requestParam) {
```

```
return Results.success(ticketService.purchaseTicketsV1(requestParam));
}
```

简单描述下这个注解的一些参数：

- scene：最开始和大家说过，组件库封了两种方案，HTTP 和消息队列，如果是 HTTP 选择使用 RESTAPI。
- type：HTTP 防重复提交有多种实现方案，如何构建唯一标识呢？SpEL 就是通过 key 参数运行时生成。
- key：填写 SpEL 表达式，运行时通过填写的 SpEL 表达式构建唯一的锁标识。
- uniqueKeyPrefix：区分业务的前缀，拼接上 key 参数用来生成真正唯一的锁标识。可不指定。
- message：满足防重复提交条件后，开始触发抛异常行为，异常中的提示信息。可不指定。

什么是 SpEL 表达式？

SpEL (Spring Expression Language) 是 Spring 框架提供的一种表达式语言，用于在运行时评估表达式。它支持在 Spring 应用程序中进行动态求值和访问对象的属性、方法调用、运算符操作等。

SpEL 表达式的语法类似于其他编程语言的表达式语言，具有以下特点：

1. 属性访问：可以使用点号 (.) 来访问对象的属性，例如 person.name。
2. 方法调用：可以通过在表达式中使用方法名和参数来调用对象的方法，例如 person.getName()。
3. 运算符操作：支持常见的算术运算符（如加减乘除）、逻辑运算符（如与、或、非）和比较运算符（如等于、大于、小于等）。
4. 条件表达式：支持条件表达式，例如三元运算符 condition ? value1 : value2。
5.

SpEL 功能很多，在这里就不一一举例说明了。接下来给大家解析下咱们这个购票接口的 SpEL 表达式是什么？

```
// 第一部分
T(org.opengoofy.index12306.framework.starter.bases.ApplicationContextHolder).getBean('environment').getProperty('unique-name', '')
```

```
// 第二部分
+ '_'+
// 第三部分
T(org.opengoofy.index12306.frameworks.starter.user.core.UserContext).getUsername()
```

简单来说，第一部分通过 Spring IOC 容器获取 unique-name 这个参数，如果为空返回空字符串，第三部分负责获取当前登录用户，第二部分负责将两个值通过 _ 字符进行拼接。

举例：你设置了 unique-name 参数为 mading，当前用户名 admin，那么最终就是 mading_admin。

验证提交参数

用户在提交购票流程时，我们需要在座位分配以及创建订单前，判断用户提交的参数以及用户是否可以购买车票。

[手摸手之实现用户购票责任链验证](#)

避免分配相同座位

假设，当大量用户抢同一个列车商务座，如果不通过技术手段加以控制，是不是就会出现相同座位分配给多个人的情况？

为了避免这一情况，我们通过 Redisson 分布式锁来解决这个问题。同一列车在同一时间下仅有单个用户可以进行座位分配以及创建订单行为，再加上 Redisson 分布式锁的自动续期等功能，可以很好保障系统运行稳定和脏数据问题。

[从根上理解Redis分布式锁演进架构](#)

座位分配

座位分配背景

当一家四口人出去玩在 12306 进行购票时，假定四个人都选择了商务座，你是想一家人被安排到各个不同的车厢还是在同一个车厢不同座位还是一个车厢相邻座位？想都不用想，肯定是后者最好对不对。那如果要实现这个座位分配算法，却不是这么容易。

先放一张高铁商务座的座位分布图，复兴号商务座两排三座，一个车厢排除空座位，共 5 个可用座位，如下所示：

复兴号 01 车厢：商务座			
01A	不可售：空座位	过道	01F
02A	02C		02F

大家也可以去看下 12306 上实际购买复兴号高铁试试，一排三个座位，分别是 ACF。咱们系统都是严格按照原系统逻辑设计的。

列车信息（以下余票信息仅供参考）

2023-09-06（周三） G8903次 北京南站（22:02开）—天津西站（22:45到）

商务座（¥177.0）17张票 一等座（¥85.0）有票 二等座（¥50.0）有票

*显示的卧铺票价均为上铺票价，供您参考。具体票价以您确认支付时实际购买的铺别票价为准。

乘客信息（填写说明）

受让人

乘车人

序号	票种	席别
1	成人票	商务座

请核对以下信息

2023-09-06（周三） G8903次 北京南站（22:02开）—天津西站（22:45到）

序号	席别	票种	姓名	证件类型	证件号码
1	商务座	成人票		中国居民身份证	

*如果本次列车剩余席位无法满足您的选座需求，系统将自动为您分配席位。

选座喽

已选座0/1

窗

A

C

过道

F

窗

本次列车，商务座余票 17 张。

返回修改

确认

提交订单表示已阅读并同意 《国铁集团铁路旅客运输规程》 《服务条款》

上一步

提交订单

座位分配难点

座位如果要想分配的合理，需要考虑的情况比较多，比如：

- 单个车厢可能容纳不下购票乘车人数；
- 单排座位可能容纳不下购票乘车人，需要根据不同座位类型拆成多排分配；
- 单个车厢可能容纳的下购票乘车人，但没有相邻座位；
- 选中座位类型车厢余量加一起都容纳不下购票乘车人的情况。

座位分配逻辑

考虑到种种情况，咱们的高铁座位分配算法流程如下，还是以商务座举例。

为了让咱们能更专注基本的座位分配算法，以下流程不包含购票人数大于等于三人（大于等于三人就要进行拆座位）以及在线选座等流程。

- 如果购票人数为两人，购买同一车厢，座位优先检索两人相邻座位并排分配。
- 假设当前正在检索的车厢不满足两人并排，就执行搜索全部满足两人并排的车厢。
- 如果搜索了所有车厢还是没有两人并排做的座位，那么执行同车厢不相邻座位。
- 如果所有车厢都是仅有一个座位，就开始执行最后降级操作，不同车厢分配。

这里生动形象的演示了几组用户购票对应的座位分配顺序，结合咱们流程查看理解更顺畅。

列车座位分配示意图

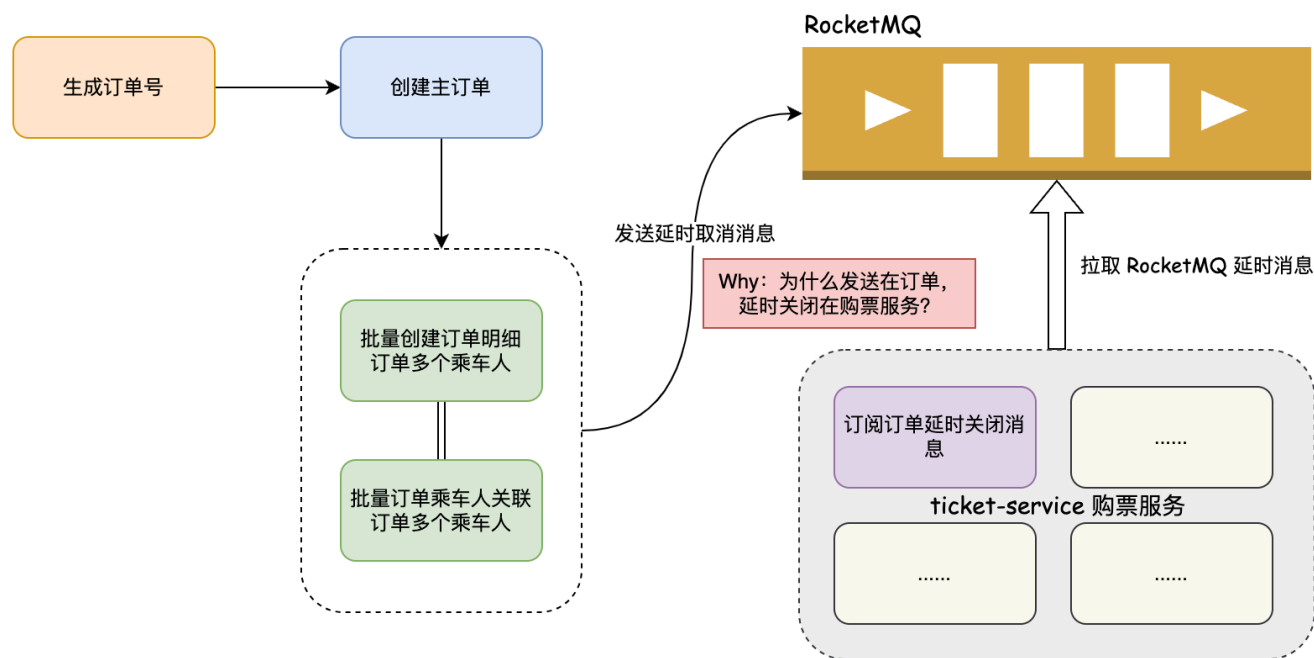
订单创建

为乘车人分配好座位后，没有问题的情况下，开始执行订单创建流程，通过 SpringCloud OpenFeign 发起远程调用。

```
TicketOrderCreateRemoteReqDTO orderCreateRemoteReqDTO =
TicketOrderCreateRemoteReqDTO.builder()
    .departure(requestParam.getDeparture())
    .arrival(requestParam.getArrival())
    .orderTime(new Date())
    .source(SourceEnum.INTERNET.getCode())
    .trainNumber(trainDO.getTrainNumber())
    .departureTime(trainStationRelationDO.getDepartureTime())
    .arrivalTime(trainStationRelationDO.getArrivalTime())
    .ridingDate(trainStationRelationDO.getDepartureTime())
    .userId(UserContext.getUserId())
    .username(UserContext.getUsername())
    .trainId(Long.parseLong(requestParam.getTrainId()))
    .ticketOrderItems(orderItemCreateRemoteReqDTOList)
    .build();
// 发起订单服务远程调用
ticketOrderResult = ticketOrderRemoteService.createTicketOrder(orderCreateRemoteReqDTO);
if (!ticketOrderResult.isSuccess() || StrUtil.isBlank(ticketOrderResult.getData())) {
```

```
log.error("订单服务调用失败，返回结果：{}", ticketOrderResult.getMessage());  
throw new ServiceException("订单服务调用失败");  
}
```

接下来给大家解析订单服务 order-service 如何操作创建订单流程。



订单号生成

手摸手之订单如何分库分表

创建订单记录

订单相关记录需要插入数据库中，有三类订单数据，分别是：主订单表、订单明细表以及订单乘车人明细表。

- 主订单：用户购买的本次订单相关信息。
- 订单明细表：用户发起购票订单请求时，可能会为多名乘车人购票，每个乘车人会有对应的订单明细记录。
- 订单乘车人明细表：[手摸手之乘车人本人车票订单查看](#)。

延迟关闭订单

用户发起订单后，如果长时间未支付，需要将订单关闭，也就是大家常说的延时关闭订单。

铁路 12306 系统中，使用 RocketMQ 作为延时关闭订单的技术实现，从整体考虑，处理能力以及稳定性相对来说较为合适。

想看关于更多延时关闭功能选型查看：[订单延时关闭功能技术选型](#)

看到上面流程图中留了个疑问，为什么延迟关闭订单发送端在订单服务，消费延迟关闭订单消息却是在购票服务？

这是为了避免服务之间的循环依赖。大家都知道，咱们列车购票是从购票服务发起的，通过购票服务分配座位、锁定座位以及创建订单，已知购票服务调用了订单服务。而延迟关闭订单中，涉及到订单状态的变更和列车座位的变更等，这样就涉及到两个服务的调用。如果消息消费在订单服务，就涉及到远程循环依赖，这对于微服务涉及中是需要尽可能避免的。所以，最终将消费延迟关闭消息放在了购票服务。

更新: 2023-08-24 11:49:24

原文: <<https://www.yuque.com/magestack/12306/nmmgqkgbfbx2bwl0>>